

Distribution Management System

Complete Technical and Functional Documentation

Version: 1.0

Date: 2026-04-07

Audience: Developers, DevOps, Product Owners, Stakeholders, and End Users

1. Overview

1.1 Introduction

The Distribution Management System is a full-stack web platform used to manage device inventory, hierarchical distribution, defect reporting, replacement processing, return workflows, approvals, and operational reporting.

It is designed for organizations that move physical devices through multiple operational tiers (for example: central distribution to sub-distributor to cluster to operator).

1.2 Objectives and Goals

- Maintain end-to-end device lifecycle visibility from registration to return/replacement.
- Enforce role-based control on who can view, create, approve, and modify records.
- Reduce manual errors in distribution by enforcing hierarchy validations.
- Track operational events through notifications, audit logs, and activity logs.
- Provide reporting, backups, and operational dashboards for management.

1.3 Problems It Solves

- Untracked device handovers and unclear ownership.
- Inconsistent defect and return handling.
- Lack of accountability in approvals and status transitions.
- Fragmented reporting and weak operational traceability.
- Difficulty managing large user hierarchies and scoped access.

1.4 High-Level Summary

The system consists of:

- React frontend for role-based workflows.
 - FastAPI backend exposing secured REST APIs.
 - MySQL database for transactional records and audit data.
 - Dockerized deployment model.
 - Cookie + JWT + CSRF authentication model.
 - Monthly automated backup generation.
-

2. System Architecture

2.1 Architecture Pattern

- Pattern: Layered client-server web application.
- Frontend: Single-page application (React + Vite).
- Backend: FastAPI service with route-service-data access layering.
- Database: MySQL with schema initialization and lightweight startup migrations.

2.2 Major Components

1. Presentation Layer (Frontend)

- Route-protected pages by role.
- Context providers for auth and notifications.
- API service layer for backend communication.

2. API Layer (Backend Routes)

- Endpoints organized by domains: auth, users, devices, distributions, defects, returns, approvals, reports, dashboard, notifications, change requests, external inventory.

3. Business Layer (Backend Services)

- Encapsulates workflow logic (distribution creation, replacement gates, return automation, approval routing, etc.).

4. Data Layer

- MySQL access via aiomysql.
- Row conversion utilities for consistent API payload shape.

5. Cross-Cutting Concerns

- CORS and CSRF middleware.
- Security headers middleware.
- Optional HTTPS enforcement middleware.
- Rate limiting with slowapi.
- Structured error handling.
- Audit and activity logging.

2.3 Data Flow (Text Diagram)

1. User authenticates in frontend.
2. Backend validates credentials and sets secure auth cookies.
3. Frontend sends API requests with credentials included.
4. Backend resolves user identity from cookie/Bearer token.
5. Route dependencies enforce role/permission checks.
6. Service layer executes business validations.
7. Database operations commit state changes.
8. Notification and activity logs are generated where applicable.

9. Frontend renders updated state.

2.4 Third-Party Integrations and Libraries

- FastAPI, Pydantic, Uvicorn
 - aiomysql
 - python-jose, passlib/bcrypt
 - slowapi
 - starlette-csrf
 - React, React Router, Tailwind CSS, Vite
 - openpyxl for spreadsheet generation/import/export
-

3. User Roles and Access Control

3.1 Role Model

Implemented normalized roles:

- super_admin
- md_director
- manager
- pdic_staff
- sub_distribution_manager
- sub_distributor
- cluster
- operator

3.2 Access Principles

- Authentication required for almost all business endpoints.
- Role checks performed both in frontend routing and backend dependencies.
- Backend remains source of truth for enforcement.
- Hierarchy-aware filters scope what non-management users can access.

3.3 Role-by-Role Access Summary

1. Super Admin

- Permissions: Full system control, role routing config updates, user management including privileged actions.
- Accessible features: All modules including approvals, reports, backups, activity feed, change request review.
- Restricted features: Cannot mutate another super admin unless self-match logic allows.
- Example: Configure approval routing for defect approvals and approve cross-functional requests.

2. MD Director

- Permissions: Read-heavy strategic access.
- Accessible features: Dashboards, reports, activities, backups, read-only access in key modules.

- Restricted features: Cannot perform mutating operations in many modules (explicit read-only protections).
- Example: Review operational KPIs and backlog trends without changing transactional data.

3. Manager

- Permissions: Broad operational management.
- Accessible features: User management scope, device registration/import, distribution operations, approvals, returns, defects, reports.
- Restricted features: Some privileged account actions reserved for super_admin.
- Example: Approve returns and monitor pending dues.

4. PDIC Staff

- Permissions: Operational management workflow participation.
- Accessible features: Devices, distributions, defects, returns, approvals, reports, external inventory.
- Restricted features: User management not equivalent to super_admin/manager.
- Example: Register incoming stock, create distribution, process approvals where routing allows.

5. Sub Distribution Manager

- Permissions: Scoped management under branch constraints.
- Accessible features: Scoped users/devices/distributions, defects, replacements, pending dues.
- Restricted features: Cannot create top-level management users.
- Example: Manage branch-level user tree and monitor branch defect/replacement status.

6. Sub Distributor

- Permissions: Mid-tier redistribution and branch visibility.
- Accessible features: My users, my devices, delivery confirmations, replacement confirmation, defects, returns.
- Restricted features: Cannot bypass hierarchy constraints when creating users/distributions.
- Example: Distribute to clusters/operators under own branch only.

7. Cluster

- Permissions: Cluster-level operations.
- Accessible features: My users/devices/distributions, defect and return operations, delivery/replacement confirmations.
- Restricted features: Cannot act outside own cluster hierarchy.
- Example: Forward devices to assigned operators and handle local defect lifecycle.

8. Operator

- Permissions: Field operations.
- Accessible features: My devices, transfer to operators in same cluster, defect creation, return tracking, delivery confirmation.
- Restricted features: Cannot access management workflows or cross-branch records.
- Example: Report defective device, receive replacement, confirm replacement transfer.

4. Complete Workflow

4.1 User Journey

1. User receives credentials from authorized creator.
2. User logs in with email/password.
3. Backend returns cookie-backed session.
4. If forced credential update flags are true, user must complete forced update flow before full app access.
5. User lands on dashboard scoped by role.
6. User operates within module permissions and hierarchy scope.

4.2 Distribution Flow

1. Sender selects recipient and devices.
2. Backend validates:
 - Sender-recipient hierarchy rules.
 - Device existence and eligibility.
 - Device not defective.
 - Device not currently locked in pending/disputed transfer chain.
3. Distribution record created with status pending_receipt.
4. Manifest file generated and stored.
5. Recipient receives notification to confirm delivery.
6. Recipient confirms:
 - If received true: status moves to approved and holder transfer is executed.
 - If received false: status moves to disputed and management/sender notified.
7. For disputed cases, management can confirm disputed return to unlock redistribution.

4.3 Approval and Acceptance Flow

1. Certain operations generate approval records.
2. Approval viewer list is filtered by role-routing configuration (approval_role_routing table).
3. Reviewer approves or rejects with optional notes.
4. Decision updates both approval record and entity state (distribution/return/defect).
5. Requester is notified.

Decision logic highlights:

- Role routing can enable/disable super_admin, manager, pdic_staff per approval type.
- A role denied by routing cannot process that approval type even if normally privileged.

4.4 Defect Reporting Flow

1. Operator/field role creates defect report for device.
2. Backend checks device exists and no active unresolved defect already exists for same device.
3. Defect status starts as reported.
4. Device status is synchronized to defective.

5. Notification targets:

- If routed via sub_distributor and conditions match, sub-distributor path used.
 - Otherwise management queue notified.
6. Defect can be forwarded to management by sub_distributor where applicable.
 7. Management updates defect status, resolves, or moves toward replacement.

Status lifecycle in implementation:

- reported
- approved or rejected
- replacement_pending_confirmation or replacement_waiting_for_device
- resolved

4.5 Replacement and Resolution Flow

1. Replacement is only allowed when defect is approved.
2. Additional gate: if linked auto return exists, return must be marked received at PDIC first.
3. Replacement source options:
 - Existing available/returned device
 - Newly registered replacement device
 - Lookup by MAC or serial
4. On replacement assignment:
 - Defect is updated with replacement metadata.
 - Old device marked replaced.
 - Notifications sent to involved users for confirmation.
5. Replacement confirmation endpoint finalizes user-facing closure path.

4.6 Return Flow

1. Return created manually or auto-created after defect approval.
2. Approval workflow processed per role-routing rules.
3. When return status becomes received:
 - Device holder reset to PDIC distribution ownership.
 - Defect payment due metadata can be updated.
4. Requester and staff receive operational notifications.

5. Features and Modules (Detailed)

5.1 Authentication and Session Management

- Inputs: email, password, refresh token, password update payloads.
- Outputs: access token/refresh token cookies, user profile payload.
- Logic:

- Failed login tracking and lockout after threshold.
- Token blacklisting on logout.
- Forced first-login credential rotation support.
- Edge cases:
- Locked accounts return HTTP 423.
- Invalid token or stale role mismatch invalidates session.
- UI behavior:
- Redirect to login if unauthenticated.
- Redirect to forced credential update page when required.

5.2 User Management

- Inputs: user profile fields, role, parent_id, status changes.
- Outputs: paginated user lists and CRUD responses.
- Logic:
- Creator role restrictions with allowed target roles.
- Branch containment checks for scoped roles.
- Special safeguards for super_admin mutation.
- Edge cases:
- Parent role mismatch rejects creation.
- Role-specific parent rules strictly validated.
- UI behavior:
- User and hierarchy pages vary by role.

5.3 Device Management

- Inputs: single/bulk registration data, status updates, edit requests.
- Outputs: device records, history timelines, availability views.
- Logic:
- Status transitions and holder updates tracked in device_history.
- Replacement pool endpoint restricts to management roles.
- Bulk upload validates file signatures and format.
- Edge cases:
- Non-management users only see held devices.
- Repair-holder endpoint supports data correction from history.
- UI behavior:
- Track page supports serial search.
- Register page includes scanner pathways.

5.4 Distribution Management

- Inputs: recipient id, device ids, notes, bulk CSV/XLSX identifiers.
- Outputs: distribution records, manifest files, export files.
- Logic:
- Hierarchy and possession checks before creation.
- Deferred holder transfer until recipient confirms receipt.
- Dispute path and admin return confirmation supported.
- Edge cases:

- Pending/disputed lock prevents duplicate transfer chains.
- Self-transfer forbidden where applicable.
- UI behavior:
- Delivery confirmation pages for recipient roles.

5.5 Defect Management

- Inputs: defect metadata, status updates, payment info, replacement actions.
- Outputs: defect records with enrichment, pending dues summaries.
- Logic:
- Duplicate active defect prevention by device.
- Route-to-sub-distributor and forward-to-management pathways.
- Auto-return creation on approved defects.
- Payment confirmation requires return received state.
- Edge cases:
- Defect replacement blocked until return receipt gate is satisfied.
- UI behavior:
- Defect list and create forms role-scoped.

5.6 Returns Management

- Inputs: return creation/status changes with optional notes and due data.
- Outputs: return records and linked approvals.
- Logic:
- Approval creation on return request.
- Received status resets ownership to PDIC.
- Defect-linked financial metadata updates when applicable.
- Edge cases:
- Cancel allowed only by requester when pending.
- UI behavior:
- Return pages for both field and management roles.

5.7 Approvals Module

- Inputs: approve/reject actions, routing config updates.
- Outputs: updated approval and entity states.
- Logic:
- Dynamic approval visibility and authority by routing table.
- Entity-aware enrichment for approval details.
- Edge cases:
- Already processed approvals are immutable.
- UI behavior:
- Approvals page visible only for approved management roles.

5.8 Notifications Module

- Inputs: system-generated events and user read/delete actions.
- Outputs: unread counts, latest notifications, paginated history.

- Logic:
- Category-specific notifications generated on key workflow events.
- Mark single/all as read operations.
- UI behavior:
- Navbar badge and notification center interactions.

5.9 Dashboard and Activities

- Inputs: role context and optional filters.
- Outputs: stats, charts, alerts, recent and admin activities.
- Logic:
- Role-scoped metric generation.
- API activity tracking persisted for meaningful business actions.
- UI behavior:
- Landing dashboard with role-appropriate KPIs.

5.10 Reports and Backup Documents

- Inputs: report type, export format, backup file uploads.
- Outputs: summaries, CSV/XLSX exports, backup document listings/downloads.
- Logic:
- Report endpoints protected for management roles.
- File name sanitization and upload size limits.
- UI behavior:
- Reports and backup pages for management.

5.11 External Inventory

- Inputs: inventory item CRUD payloads, image uploads, stock adjustments, PO receipts.
- Outputs: inventory dashboards, movements, receipts, purchase orders.
- Logic:
- Management-only write operations.
- Any-role read operations on selected views.
- Bulk CSV import with strict required columns.
- UI behavior:
- Dedicated external inventory page with operational views.

5.12 Change Requests

- Inputs: email/password/device status/transfer-fix change requests.
- Outputs: pending queues for managers/admins and review outcomes.
- Logic:
- Role-restricted submission types.
- Review path supports approve/reject with optional override values.
- Transfer-fix requests validated against defect state and operator involvement.
- UI behavior:
- Change request review page for management roles.

6. Database Design

6.1 Core Entities

- users
- devices
- device_history
- distributions
- defects
- returns
- approvals
- notifications
- change_requests
- operators
- external_inventory_items
- inventory_purchase_orders
- inventory_po_lines
- inventory_receipts
- inventory_receipt_lines
- inventory_stock_movements
- api_activity_logs
- approval_role_routing
- token_blacklist

6.2 Relationship Highlights

- users is parent table for hierarchy via parent_id.
- distributions links from_user_id and to_user_id with device_ids list.
- defects references device_id and may link to returns via defect_id or auto_return_id.
- approvals references entity_id plus approval_type.
- notifications tied to user_id.
- device_history tracks movement and state transitions by device_id.

6.3 Key Fields by Domain

1. users

- role, status, parent_id, force_email_change, force_password_change, failed_login_attempts, locked_until

2. devices

- serial_number, mac_address, status, current_holder_id, current_holder_type, metadata

3. distributions

- distribution_id, device_ids, status, from_user_id, to_user_id, manifest_file

4. defects

- report_id, status, return_amount, payment_confirmed, replacement_device_id, auto_return_id

5. returns

- return_id, status, requested_by, return_to, defect_id, received_date

6. approvals

- approval_type, entity_id, status, approved_by, rejection_reason

6.4 Schema Evolution

- Startup includes lightweight ALTER migrations for backward compatibility.
 - Default routing rows are inserted for approval types.
 - Legacy plaintext values in change_requests.new_password are migrated to bcrypt hashes.
-

7. Security and Authentication

7.1 Authentication Method

- Primary: JWT tokens delivered as HttpOnly cookies.
- Secondary support: Bearer token in Authorization header.

7.2 Authorization Logic

- Route dependencies enforce role-level access.
- Service-level checks enforce hierarchy and workflow gates.
- Forced credential update gate blocks access to most routes until completion.

7.3 Data Protection and Security Controls

- Password hashing with bcrypt.
- CSRF middleware with cookie/header token strategy.
- Security response headers (X-Frame-Options, CSP, HSTS in production, etc.).
- Rate limiting on selected auth endpoints.
- Token blacklist for logout invalidation.
- Audit logger for sensitive events.

7.4 Vulnerabilities Mitigated

- Brute-force login: failed-attempt lockout window.
- Session reuse after logout: blacklist checks.
- CSRF on unsafe methods: CSRF token validation.
- Directory traversal for uploads: safe path resolution.
- Insecure direct access: role checks + branch checks.

7.5 Important Operational Security Notes

- Production requires HTTPS because secure cookies are enabled in production behavior.
 - SECRET_KEY must be strong and unique per environment.
 - CORS origins must exactly match deployed frontend origin.
-

8. API Design

8.1 API Base

- Prefix: /api
- Style: REST-style JSON endpoints
- Standard success envelope:
 - success
 - message
 - data
- optional pagination

8.2 Major Endpoint Groups

1. Authentication

- POST /api/auth/login
- POST /api/auth/logout
- POST /api/auth/refresh
- GET /api/auth/me
- PUT /api/auth/password
- POST /api/auth/complete-forced-update

2. Users

- GET /api/users
- GET /api/users/{user_id}
- POST /api/users
- PUT /api/users/{user_id}
- DELETE /api/users/{user_id}
- PATCH /api/users/{user_id}/status
- PATCH /api/users/{user_id}/credentials

3. Devices

- GET /api/devices
- GET /api/devices/available
- GET /api/devices/for-replacement
- GET /api/devices/my-overview
- GET /api/devices/track/{serial_number}
- GET /api/devices/{device_id}
- GET /api/devices/{device_id}/history
- POST /api/devices/bulk-upload
- POST /api/devices
- PUT /api/devices/{device_id}
- PATCH /api/devices/{device_id}/status
- DELETE /api/devices/{device_id}

4. Distributions

- GET /api/distributions
- POST /api/distributions
- POST /api/distributions/bulk-upload
- POST /api/distributions/{distribution_id}/receipt
- POST /api/distributions/{distribution_id}/confirm-return
- PATCH /api/distributions/{distribution_id}/status
- GET /api/distributions/{distribution_id}/manifest
- GET /api/distributions/{distribution_id}/export-mac-nuid
- DELETE /api/distributions/{distribution_id}

5. Defects

- GET /api/defects
- POST /api/defects
- PATCH /api/defects/{defect_id}/status
- PATCH /api/defects/{defect_id}/resolve
- POST /api/defects/{defect_id}/replace
- POST /api/defects/{defect_id}/replacement/confirm
- POST /api/defects/{defect_id}/forward-to-management
- POST /api/defects/{defect_id}/confirm-payment
- GET /api/defects/pending-dues/users
- GET /api/defects/pending-dues/users/{user_id}
- GET /api/defects/pending-dues/me

6. Returns

- GET /api/returns
- POST /api/returns
- PATCH /api/returns/{return_id}/status
- DELETE /api/returns/{return_id}

7. Approvals

- GET /api/approvals
- GET /api/approvals/{approval_id}
- POST /api/approvals/{approval_id}/approve
- POST /api/approvals/{approval_id}/reject
- GET /api/approvals/role-routing/config
- PUT /api/approvals/role-routing/config

8. Dashboard, Reports, Notifications, External Inventory, Change Requests

- Dedicated route groups for each module with role-protected access.

8.3 Sample Request and Response

Example: Login request

- Request body:

- email: user@example.com
- password: passwordValue
- Success response data includes:
- access_token
- refresh_token
- token_type
- expires_in
- user object

8.4 Error Handling Contract

Global middleware returns structured error payloads:

- success false
- message
- error.code
- error.details

Validation errors return HTTP 422 with field-level details list.

9. UI and UX Structure

9.1 Core Layout

- Persistent sidebar for navigation.
- Top navbar for search/profile/notifications.
- Breadcrumb bar for route context.
- Main content outlet by route.

9.2 Main Pages

- Login, Forced Credential Update
- Dashboard
- Devices, Register Device, Track Device, Bulk Import
- Distributions, Create Distribution, Delivery Confirmations, Bulk Distribution Upload
- Defect Reports, Create Defect, Replacements, Pending Replacements, Replacement Confirmation, Pending Dues
- Returns
- Users, User Hierarchy
- Approvals
- Reports, Backup, Activities
- External Inventory
- Notifications

- Change Requests
- Profile, Settings

9.3 Navigation Behavior

- Sidebar menus rendered dynamically by normalized role.
- Child menu expansion for grouped domains.
- Access-denied routing directs to Unauthorized page.
- Unauthenticated users are redirected to Login.

9.4 Key UI Interactions

- Real-time unread notification count and latest list.
- Role-specific action buttons hidden/shown by permissions.
- Scanner-assisted device registration workflows.
- Export/download interactions for manifests and reports.

10. Error Handling and Edge Cases

10.1 System-Level Errors

- Unhandled exceptions are normalized to generic 500 responses.
- Backend logs exception stack traces server-side.

10.2 User-Level Validation Errors

- Invalid payloads return 422 with field messages.
- Business rule violations return 400/403 with meaningful details.

10.3 Workflow Fail-Safes

- Distribution lock prevents duplicate in-flight transfer conflicts.
- Defect replacement gated by return receipt to prevent premature replacement.
- Duplicate active defect prevention per device.
- Approval re-processing prevented for non-pending requests.

10.4 Typical Edge Cases Covered

- Operator attempting cross-cluster transfer.
- Sub-role creating user outside branch hierarchy.
- Replacement using same device as defective unit.
- Payment confirmation before return receipt.
- Invalid file signatures for bulk upload.

11. Deployment and Infrastructure

11.1 Runtime Topology

Docker Compose services:

- mysql (port 3306)
- backend (port 8080)
- frontend (port 5173)

11.2 Environment Setup

Backend critical values:

- ENVIRONMENT
- DEBUG
- DB_HOST/DB_PORT/DB_USER/DB_PASSWORD/DB_NAME
- SECRET_KEY
- CSRF_COOKIE_SECURE
- CORS_ORIGINS
- ADMIN_INITIAL_PASSWORD

Frontend critical value:

- VITE_API_URL (must be correct at build time)

11.3 Persistence

- MySQL data via Docker volume mysql_data.
- Mounted backend folders:
 - distribution_manifests
 - monthly_backups
 - uploads

11.4 CI/CD Status

- No explicit CI/CD pipeline definition found in repository root by default.
- Current deployment approach is Docker Compose driven.
- Recommended next step: add pipeline for build-test-scan-deploy.

11.5 Production Hosting Guidance

- Use reverse proxy or private network entry with HTTPS.
- Keep MySQL non-public when possible.
- Use secrets manager for production secrets.

12. Logging and Monitoring

12.1 Logging Strategy in Implementation

1. Audit logging

- Dedicated logger writes to backend/logs/audit.log.
- Used for security-sensitive events (login failures, password changes, DB reset attempts).

2. API activity logging

- Meaningful business API actions persisted to api_activity_logs table.
- Includes actor, method, path, status, description, IP, timestamp.

3. Application logging

- Route and service exceptions logged through Python logging.
- Frontend dev logs exist for troubleshooting API/context actions.

12.2 Monitoring Data Sources

- Container logs via docker compose logs.
- audit.log file.
- api_activity_logs table for operational timeline.
- health endpoint at /health.

12.3 Alerting

- Explicit external alert system integration not found by default.
- Recommended additions:
- Uptime probe alerts for frontend/backend.
- Error rate and latency alerts from API metrics.
- Disk usage and backup failure alerts.

12.4 Backup Monitoring

- Monthly backup scheduler runs inside backend app lifecycle.
- Marker file prevents duplicate monthly runs.
- Outputs written under backend/monthly_backups/YYYY-MM.

13. Future Improvements

13.1 Scalability

- Move from single backend instance to horizontally scaled API replicas.
- Externalize background scheduling to dedicated worker/cron service.
- Normalize JSON-heavy fields into relational tables where query pressure rises.

13.2 Reliability

- Add transactional boundaries for multi-entity workflow updates where partial failure risk exists.
- Add retry/outbox mechanism for notification creation.
- Introduce idempotency keys for sensitive create operations.

13.3 Security

- Add MFA for privileged roles.
- Add device/session management screen for account sessions.
- Enforce stricter password lifecycle and rotation policies.

13.4 Observability

- Add centralized logs (ELK/OpenSearch/Grafana stack).
- Add OpenTelemetry tracing.
- Add metrics endpoint and dashboards.

13.5 Product and UX

- Add guided onboarding for first-time users by role.
- Add richer audit explorer with export filters.
- Improve conflict resolution UI for disputed distributions.

13.6 Engineering Process

- Add automated test coverage for critical workflow transitions.
- Add CI pipeline with lint, unit tests, security scan, and release artifact tagging.

Appendix A: Platform and Stack Summary

Software Name: Distribution Management System

Purpose: Hierarchical device distribution and lifecycle management

Target Users: Super Admin, MD Director, Manager, PDIC Staff, Sub Distribution Manager, Sub Distributor, Cluster, Operator

Platform: Web application, Dockerized backend/frontend services, MySQL data store

Tech Stack: React, Vite, Tailwind CSS, FastAPI, aiomysql, MySQL, JWT, CSRF middleware, Docker Compose

Key Features: User hierarchy, device lifecycle, distribution confirmations, defect and replacement workflows, return approvals, reporting, backups, notifications, activity logging

Appendix B: Assumptions and Boundaries

- This document is generated from current repository implementation and observed runtime configuration.
- If your team maintains environment-specific overrides outside this repository, treat those as deployment-layer supersets.
- Any policy, legal, or compliance controls should be appended by your internal governance team.